I2C_Architectures.odt
by CmdrZin
www.gameactive.org
cmdrzin@yahoo.com
Created: 26apr26
Revised: 13may26

I2C Architectures

Which architecture to use is based on the operations performed by your device.

Controlling how the device response to the first byte of a write message (SDA_W data) is controlled by the module code at the state entered when the first data byte is received after a SDA_W is detected.
(see case TWI_SRX_DATA_ACK is twiRegSlave.c)

General: Reading register 0 should always returns 0x00. This is compatible with current scanning applications.
SDA_W 0x00 SDA_R <0x00>

0. Non-registered – Data used as is. Used for simple command protocols where the data directly controls some element or reads only one type of data. A simple relay control or temperature sensor.
Example: PCF8574A – Data goes to or comes from 8-bit I/O port. I_{OH} = -1ma, I_{OL} = 25ma. ONLY sink LEDs.
SDA_W data
SDA_R <data>
Example AHT20 – Data writes are commands. Data reads are streams.
SDA_W Trigger data1 data2
SDA_R State data1 data2 … CRC

1. FIFO – Data stream is parsed by main control process.
SDA_W data0 data1 … Data goes into Input FIFO to be parsed by device.
SDA_R <data0> <data1> … Data delivered from device output FIFO to be parsed by Host.
Output data is loaded into the FIFO txBufffer. SDA_R operations read successive bytes from the buffer. if the txHead does not equal the txTail, the buffer has data to transmit. If incrementing the txHead would cause it to equal the txTail, the buffer is full and will not accept more data.
Input data is loaded into the rxBuffer. The main() can test for received data by checking the rxHead and rxTail. If they are equal, then no data is available. The rxBuffer data normally copied into another buffer for parsing of messages.

2. Register Message – First byte is the register number. Successive bytes are saved to buffer or read from the buffer for the register. Upon receiving a register value, the value is used to index a table of buffer pointers for receive and transmit. The indexed pointers are then assigned to the general receive and transmit pointer along with the max size values for the buffers. An example data struct is show below.

```
typedef struct {
    volatile void* rxBuff;          // can be modified by ISR
    volatile uint8_t  rxSize;       // can be modified by ISR
    void* txBuff;
    uint8_t  txSize;
} COMM_BUFF;
```

IN WORK

It is used to build a table of locations to write to and read from based on the register number. A pointer to the location and its size are place into a table whose reference is sent to the driver.
Example:

```
COMM_BUFF rxtxBuffers[] = {
    { &statusLED, 1, &zeroBuff, 1 },            // rx 0x00   tx 0x00
    { &ledDelay, 2, &statusBuff, 1 },           // rx 0x01   tx 0x01
    { &clearCount1, 1, &count1, 4},             // rx 0x02   tx 0x02
    { &clearCount2, 1, &count2, 1},             // rx 0x03   tx 0x03
    { &count1Dir, 1, &dummyBuff, 1},            // rx 0x04   tx N/A
    { &count2Dir, 1, &dummyBuff, 1},            // rx 0x05   tx N/A
    { &setCount1, 4, &dummyBuff, 1}             // rx 0x06   tx N/A
};
// NOTE: buffer size could be replace with sizeof(buffer)
```

This is sent to the init method of the driver.

```
twiRegSlaveInit( SLAVE_ADRS, rxtxBuffers, sizeof(rxtxBuffers)/sizeof(COMM_BUFF) );
```

Data types and buffers longer than one byte have to be disassembled and reassembled in byte-order.
(see Arduino_I2C_2_demo code for an example)

3. Register List – First byte is the starting index for both the rxBuffer and the txBuffer. Successive bytes are stored or read from successive locations in the given buffer. Each buffer can be treated as a list of registers.
Example: Parallax L3G4200D Gyroscope Module. Limited to two successive read/writes.

SDA_W Register data
SDA_R Register <data>

APPENDIX I - Example Code implementing various I2C architectures

The following code has been tested on the ATtiny212, 412, 814, 1616, & 3216. It should work for any device in the tinyAVR® 1-series family.

I2C_2_Demo	Arduino tester and MPLABX project using twiRegSlave2 using rx/tx registers.
twiRegSlave3	Embedded registers with notification

IN WORK